

Dynamic Tree Queries

Topic.

Dynamic trees, binary lifting, LCA, subtree maximum queries.

Statement.

Design a data structure for a tree that supports the following operations:

- add a new leaf to the tree;
- find $\text{LCA}(a, b)$;
- find the most valuable vertex in a subtree;
- find the distance between two vertices.

Solution.

We maintain dynamic binary lifting for LCA, the distance from the root, the Euler traversal interval of each subtree, and a segment tree for the maximum value in a subtree.

The tree changes only by adding a new leaf.

We store:

$\text{parent}[v]$

the parent of v ,

$\text{depth}[v]$

the depth of v ,

$\text{dist}[v]$

the distance from the root to v with edge weights taken into account,

$\text{val}[v]$

the value of the vertex,

$\text{up}[v][k]$

the 2^k -th ancestor of v ,

$\text{tin}[v], \text{tout}[v]$

the subtree interval in the order of entry.

In the segment tree, at position $tin[v]$ we store

$$(val[v], v),$$

so that we can take the maximum in a subtree.

Operation add(p, v, w).

We attach a new leaf v to p by an edge of weight w :

$$parent[v] = p,$$

$$depth[v] = depth[p] + 1,$$

$$dist[v] = dist[p] + w.$$

Set

$$up[v][0] = p,$$

and then fill all other values $up[v][k]$.

Set

$$tin[v] = ++timer, \quad tout[v] = tin[v].$$

Put

$$(val[v], v)$$

into the segment tree at position $tin[v]$.

After adding the leaf, we extend $tout$ of all ancestors of p , and also of p itself, up to the new value of the timer.

Operation LCA(a, b).

We find LCA in the standard way using binary lifting in

$$O(\log n).$$

Operation getMax(u).

The subtree of u is the segment

$$[tin[u], tout[u]].$$

The answer is the maximum on this segment in the segment tree.

Operation $\text{dist}(a, b)$.

Let

$$l = \text{LCA}(a, b).$$

Then

$$ans = dist[a] + dist[b] - 2dist[l].$$

Complexity.

The operations LCA, dist, and getMax work in

$$O(\log n).$$

The operation add takes

$$O(\log n)$$

for filling binary lifting and updating the segment tree, and also

$$O(h)$$

for updating $tout$ of the ancestors.

Therefore,

$$\text{add} = O(\log n + h),$$

and in the worst case

$$O(n).$$